```python
# Install libraries
!pip install torch torchvision pillow matplotlib

# Imports
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models, transforms
from PIL import Image
import matplotlib.pyplot as plt
import copy
from google.colab import files
import os

# Upload images
print("Upload 2 images: first = content image, second = style image")
uploaded = files.upload()

# Get uploaded file names
file_names = list(uploaded.keys())
content_path = file_names[0]
style_path = file_names[1]

print("Content image:", content_path)
print("Style image:", style_path)

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# Image loader
def load_image(path, size=512):
    image = Image.open(path).convert("RGB")
    transform = transforms.Compose([
        transforms.Resize((size, size)),
        transforms.ToTensor()
    ])
    image = transform(image).unsqueeze(0)
    return image.to(device)

content_img = load_image(content_path)
style_img = load_image(style_path)

# Normalization for VGG
class Normalization(nn.Module):
    def __init__(self, mean, std):
        super().__init__()
        self.mean = torch.tensor(mean).view(-1,1,1).to(device)
        self.std = torch.tensor(std).view(-1,1,1).to(device)
```

```
    def forward(self, img):
        return (img - self.mean) / self.std

# Load pretrained VGG19
cnn =
models.vgg19(weights=models.VGG19_Weights.DEFAULT).features.to(device)
.eval()

for param in cnn.parameters():
    param.requires_grad = False

normalization_mean = [0.485,0.456,0.406]
normalization_std = [0.229,0.224,0.225]
normalization = Normalization(normalization_mean, normalization_std)

# Content & style layers
content_layers = ['conv_4']
style_layers = ['conv_1','conv_2','conv_3','conv_4','conv_5']

# Gram matrix
def gram_matrix(input):
    b,c,h,w = input.size()
    features = input.view(c, h*w)
    G = torch.mm(features, features.t())
    return G.div(c*h*w)

# Content loss
class ContentLoss(nn.Module):
    def __init__(self,target):
        super().__init__()
        self.target = target.detach()

    def forward(self,input):
        self.loss = nn.functional.mse_loss(input,self.target)
        return input

# Style loss
class StyleLoss(nn.Module):
    def __init__(self,target_feature):
        super().__init__()
        self.target = gram_matrix(target_feature).detach()

    def forward(self,input):
        G = gram_matrix(input)
        self.loss = nn.functional.mse_loss(G,self.target)
        return input

# Build model
def get_model_and_losses(cnn, normalization, content_img, style_img):
```

```
    cnn = copy.deepcopy(cnn)

    content_losses = []
    style_losses = []

    model = nn.Sequential(normalization)
    i = 0

    for layer in cnn.children():

        if isinstance(layer, nn.Conv2d):
            i += 1
            name = f'conv_{i}'
        elif isinstance(layer, nn.ReLU):
            name = f'relu_{i}'
            layer = nn.ReLU(inplace=False)
        elif isinstance(layer, nn.MaxPool2d):
            name = f'pool_{i}'
        else:
            name = f'layer_{i}'

        model.add_module(name, layer)

        if name in content_layers:
            target = model(content_img).detach()
            content_loss = ContentLoss(target)
            model.add_module("content_loss_"+str(i),content_loss)
            content_losses.append(content_loss)

        if name in style_layers:
            target_feature = model(style_img).detach()
            style_loss = StyleLoss(target_feature)
            model.add_module("style_loss_"+str(i),style_loss)
            style_losses.append(style_loss)

    return model, style_losses, content_losses


model, style_losses, content_losses = get_model_and_losses(
    cnn, normalization, content_img, style_img
)

# Optimization
input_img = content_img.clone()
optimizer = optim.LBFGS([input_img.requires_grad_()])

style_weight = 30000
content_weight = 25

num_steps = 300
```

```python
run = [0]

while run[0] <= num_steps:

    def closure():

        input_img.data.clamp_(0,1)
        optimizer.zero_grad()

        model(input_img)

        style_score = 0
        content_score = 0

        for sl in style_losses:
            style_score += sl.loss

        for cl in content_losses:
            content_score += cl.loss

        loss = style_weight*style_score + content_weight*content_score
        loss.backward()

        if run[0] % 50 == 0:
            print("Step:",run[0])

        run[0] += 1
        return loss

    optimizer.step(closure)

input_img.data.clamp_(0,1)

# Show result
def imshow(tensor):
    image = tensor.cpu().clone().detach().squeeze(0)
    image = image.permute(1,2,0)

    plt.figure(figsize=(8,8))
    plt.imshow(image)
    plt.axis("off")
    plt.show()

imshow(input_img)
```

```
Requirement already satisfied: torch in
/usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in
/usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: pillow in
/usr/local/lib/python3.12/dist-packages (11.3.0)
```

```
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from torch) (3.25.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4-
>torch) (1.4.0)
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7-
>matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch)
(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Upload 2 images: first = content image, second = style image

<IPython.core.display.HTML object>

Saving Style.webp to Style (1).webp
Saving Content.jpg to Content (3).jpg
Content image: Style (1).webp
Style image: Content (3).jpg
Using device: cuda
Downloading: "https://download.pytorch.org/models/vgg19-dcbb9e9d.pth"
to /root/.cache/torch/hub/checkpoints/vgg19-dcbb9e9d.pth

100%|██████████| 548M/548M [00:03<00:00, 158MB/s]

Step: 0
Step: 50
Step: 100
Step: 150
Step: 200
Step: 250
Step: 300
```